



Hebi-Test-Software

Codebase Guide

A quick reference for understanding the project structure, where the main logic lives, and what to edit when changes are needed.

Purpose: This guide turns the original code explanation into a cleaner onboarding document for future edits, maintenance, and project handoff.

1. Project Overview

At a high level, the application is organized into three layers: startup, user interface, and hardware/service logic. The startup file launches the application, the UI file handles screens and user actions, and the HEBI service file manages actuator communication and test behavior.

Simple mental model

- **main.py** starts the application.
- **wheel_test_app/main_window.py** builds the interface and reacts to the user.
- **wheel_test_app/hebi_service.py** communicates with the actuator and computes test behavior.
- **README.md** explains setup, usage, workflow, and outputs.
- **requirements.txt** defines the required Python packages.

2. Core Files and Their Roles

Use the table below as a quick map of the codebase. It highlights what each file is for and when you would normally edit it.

File	Main purpose	Edit when...
README.md	Project guide: setup, execution, workflow, CSV output, and general usage notes.	The user workflow, setup, outputs, or main features change.
requirements.txt	List of Python packages installed with pip.	A new dependency is added or removed.
run_app.ps1	Launcher that runs the app with the local virtual environment Python.	The startup process or execution path changes.
main.py	Entry point that starts the application and shows a friendly package-missing message.	You need to change how the application starts.
main_window.py	Main UI file. Contains the screens, controls, styling, timers, test flow, and	You are changing layout, widgets,



	export flow on the interface side.	checklist items, colors, or UI-side CSV/export behavior.
hebi_service.py	Hardware/service layer. Contains HEBI-related logic, telemetry structures, test plans, and actuator communication.	You are changing direction mapping, movement logic, discovery, HEBI commands, or telemetry handling.
__init__.py	Package boilerplate.	Rarely — usually only for package-level adjustments.
.gitignore	Defines files and folders that should not be committed, such as .venv and cache files.	You want to ignore additional generated files or export folders.

3. Main UI File: `main_window.py`

This is the file you will probably edit most. It contains the first screen, the single-wheel test interface, styling, timers, and the UI side of the CSV export flow.

Main responsibilities

- The first screen where the user chooses 1 wheel or 2 wheels.
- Logo loading and initial navigation.
- The full single-wheel test interface.
- Buttons, styles, checklist items, inputs, and telemetry view.
- CSV export handling on the UI side.
- Timers for test polling and zero-position polling.

Key classes and methods

- **WheelChoicePage** First screen with the logo and wheel selection.
- **SingleWheelTestPage** Main working test page.
- **_build_configuration_panel()** Builds the left-side configuration inputs.
- **_build_runtime_panel()** Builds the right-side runtime and telemetry area.
- **_update_derived_fields()** Updates direction, duration, and deployment/excavation behavior when the selected mode changes.
- **_start_test()** Starts excavation or deployment logic.
- **_poll_test()** Runs repeatedly during the test and updates telemetry.
- **_finish_test()** Stops the test and handles export state.
- **_export_pending_deployment_data()** Writes deployment CSV data only after the mass is entered.
- **_zero_position()** / **_poll_zero_move()** Moves the actuator to 0 rad and monitors the move.
- **_apply_styles()** Defines theme colors and widget appearance.



Typical UI-side changes

- **Layout or widgets:** Edit `main-window.py`.
- **Colors or theme:** Edit `_apply_styles()` in `main_window.py`.
- **Checklist items:** Edit `LAB_CHECKLIST_ITEMS` in `main-window.py`.
- **CSV export behavior on the interface side:** Mostly edit `main_window.py`.

4. Hardware and Service Layer: `hebi_service.py`

This file should hold the **HEBI-specific logic, not the visual interface**. It is the place to edit communication, direction logic, velocity behavior, and feedback handling.

Main responsibilities

- Direction mapping for excavation and deployment.
- The `TestPlan` dataclass.
- The `TelemetrySample` dataclass.
- HEBI motor discovery on the network.
- Connection to the selected actuator.
- Sending and refreshing velocity commands during a test.
- Sending the 0 rad position command.
- Reading actuator feedback.
- Generating preview feedback when hardware is not connected.

Key functions and data structures

- `MODE_TO_DIRECTION` Defines the sign and label for each mode.
- `build_test_plan(...)` Converts user inputs into a validated test plan.
- `discover_modules()` Finds available HEBI motors.
- `connect(...)` Connects to a selected motor.
- `start_velocity_test(...)` Sends the first motion command.
- `refresh_velocity_command(...)` Keeps the actuator moving during the test.
- `zero_position(...)` Sends the command to move to 0 rad.
- `read_feedback(...)` Gets real telemetry.
- `preview_feedback(...)` Provides simulated telemetry when hardware is not connected.

Typical service-side changes

- Direction sign or mode mapping.
- Movement equations or validated plan logic.
- Motor discovery or connection behavior.
- HEBI communication details.
- The telemetry values that are collected or exposed to the UI.

5. Recommended Editing Workflow

1. Start with the file map table to identify where the change belongs.
2. If the change affects what the user sees or clicks, inspect `main_window.py` first.
3. If the change affects actuator behavior or telemetry, inspect `hebi_service.py` first.
4. If the change affects setup, dependencies, or how the app is launched, update `README.md`, `requirements.txt`, or `run_app.ps1` as needed.

- When user workflow or outputs change, update README.md so the project guide stays aligned with the software.

Note. This guide is intended as a practical orientation document. It does not replace reading the code itself, but it gives a reliable starting point for deciding where to look and what to edit.

6. Appendices

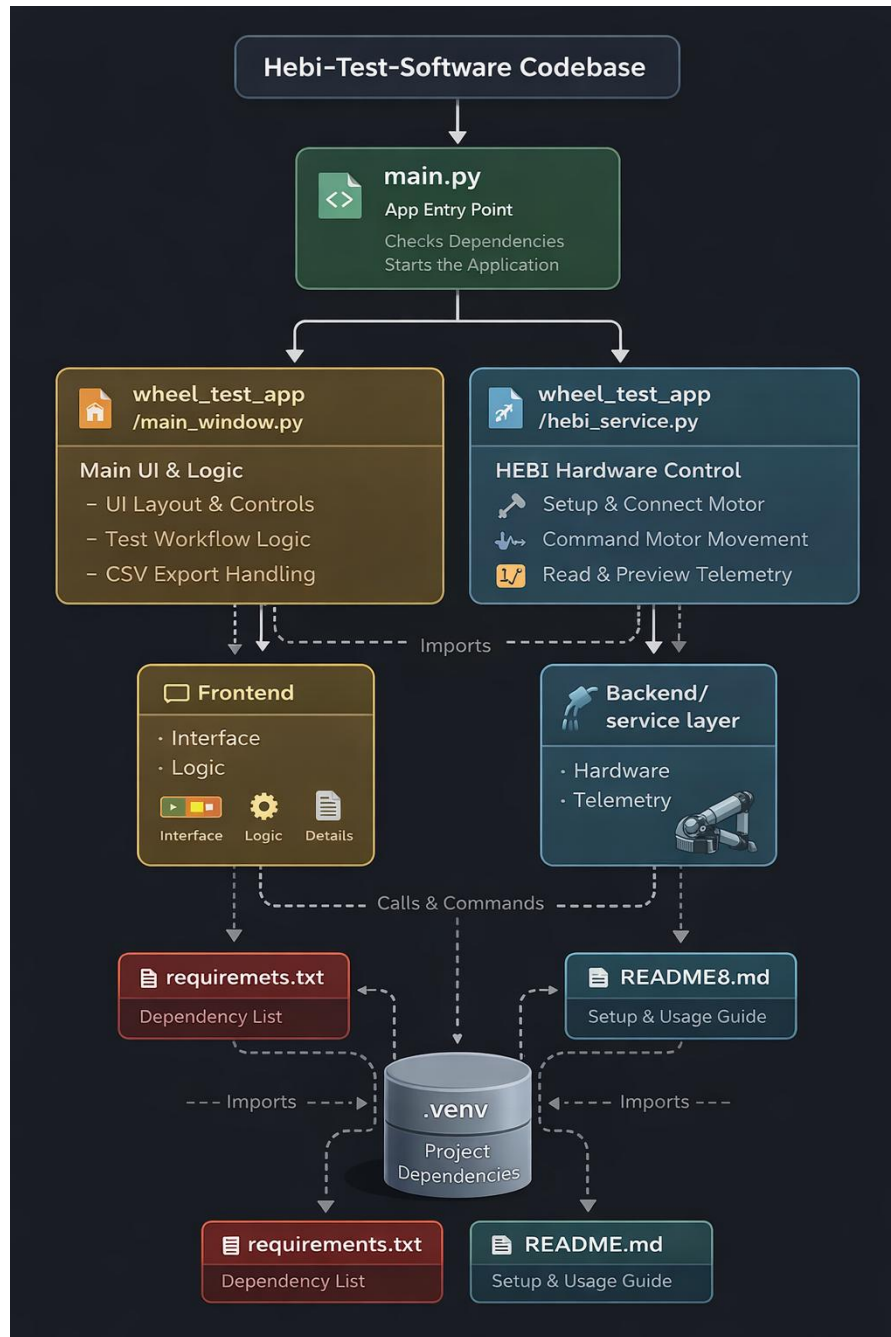


Fig 1. Block Diagram. Made by ChatGPT from the Github repository.

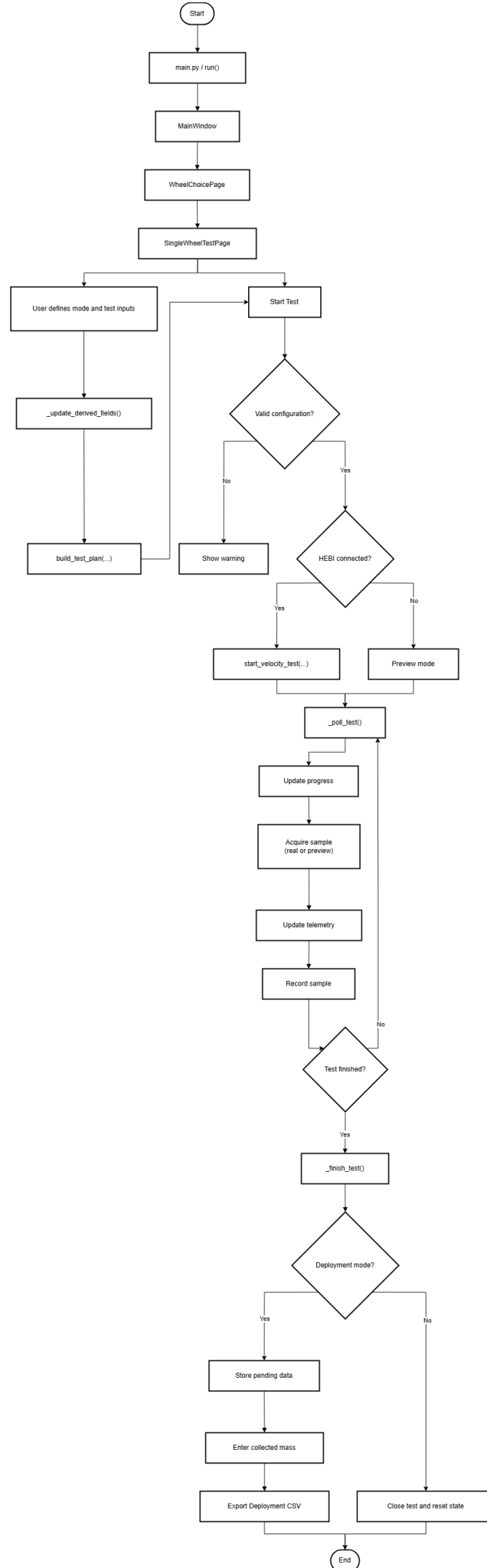


Fig 2. Flow chart of the software. Own elaboration